

バージョン管理と同時開発のしくみ

なぜ Git で管理するのか / 同時に直したら何が起きるのか — アプリマネージャーの裏側を仕組みから理解する

1. なぜ Git か

共有フォルダの限界と履歴管理

2. 運用のかたち

用語・対応表・ライフサイクル

3. 同時開発

提出時とリリース時に起こること

4. β 版基点の開発

スタックの記録とリスク

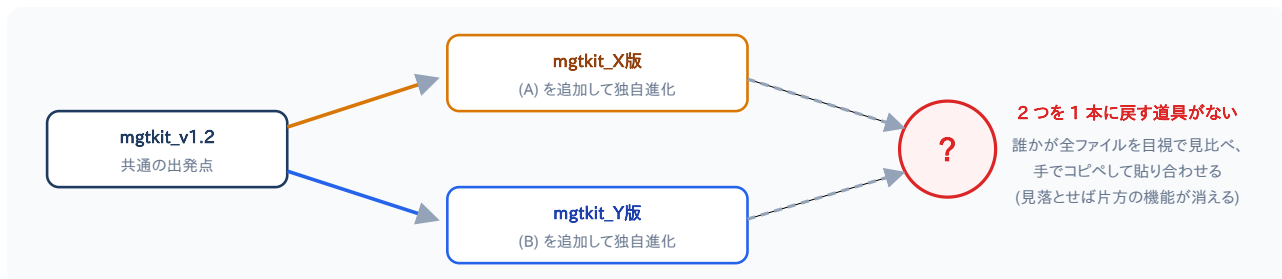
5. プラクティス

結局どうすべきか

1 なぜ Git で管理するのか

1.1 Dropbox でも「工夫」はできる — それでも残る合流の壁

フォルダを人ごと・版ごとに分ければ上書き事故や競合コピーは減らせますし、Dropbox にも版履歴・復元機能はあります。問題は保存ではなく、分けたフォルダを「元の 1 本に混ぜ直す」工程 — ここを支援する仕組みが存在しないことです。



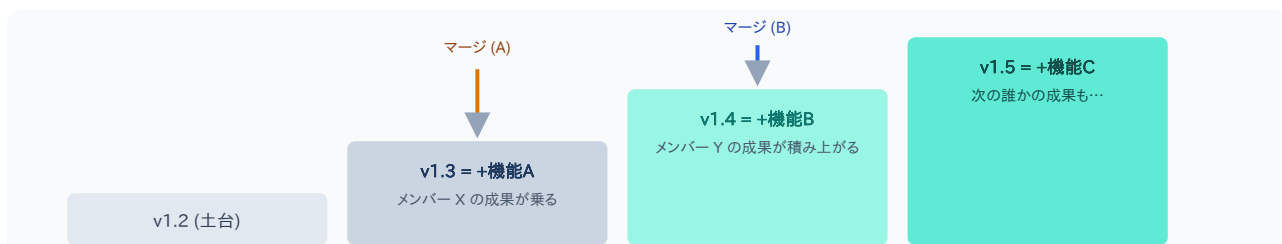
フォルダを分けた瞬間、版は分岐する。分岐を安全に「合流」させる工程だけが、永遠に手作業のまま残る

フォルダ分け運用でも残る 3 つの限界

- 合流が全手作業 — 2 つのフォルダの差を目視で洗い出し、行単位の取捨選択を人間が全ファイルで行う。並行数×人数に比例して破綻する
- 「共通の出発点」の記録がない — どの版から分かれたかを覚えていないので、「相手が消した行」と「自分が足した行」の区別すら機械的にできない
- 機能単位の履歴と検証ゲートがない — 履歴はファイル単位の自動保存で「この 5 ファイルで 1 機能・理由はこれ」が残らず、テスト・承認をってから反映という関門も作れない

1.2 Git の本領 — 「合流」が仕組み化され、機能が積み上がる

Git は最初から「分岐して、合流する」ために設計された道具です。全員の共通の出発点 (共通祖先) を必ず記録しているため、マージ時は 3 点比較で自動合成でき、判断が要る箇所 (同じ行) だけを人間に問い合わせます。



合流のたびに幹へ機能が積み上がる。並行して作られた成果が、失われずに 1 本の製品へ蓄積されていく

複数人開発での Git の強み — ここが本質

- 機能が積み上がる — 全員の成果がマージで幹 (main) に合流し、v1.2 → v1.3 → v1.4 と雪だるま式に蓄積される。フォルダ分け運用の「どれが最新? 誰の版に何が入ってる?」が構造的に消える
- 合流コストがほぼ一定 — 共通祖先を知る 3-way マージが自動合成するため、2 人でも 10 人でも合流の手間はほぼ変わらない。人数が増えるほどフォルダ運用との差が開く
- 積む前に検査でき、段ごとの記録が残る — 各段の手前に CI + 2 人の承認のゲート。コミットには「誰が・なぜ」が残り、段単位の巻き戻し (revert) と原因行の特定 (blame) ができる

2 mgtkit での Git 運用のかたち

2.1 用語ミニ辞典 — 6 語だけ覚えれば全部読める

コミット commit

変更の最小単位。「誰が・いつ・何を・なぜ」を持つ保存点。SHA で一意に指せる



ブランチ branch

履歴の枝分かれ。作業を隔離して、幹 (main) を検証済みだけに保つ



マージ merge

枝の合流。共通祖先と両者の 3 点を見比べて自動合成 (3-way マージ)



プルリクエスト pull request / PR

「この枝を幹に入れて」というレビュー申請。差分・承認・CI がぶら下がる



クローン clone

履歴ごとリポジトリを複製。setup.bat の初回がこれ。全員が完全な写しを持つ



フォーク fork

他人のリポジトリを自分のアカウントへ複製。mgtkit では通常使わない (第三者が PR を送る経路。承認は数えない)

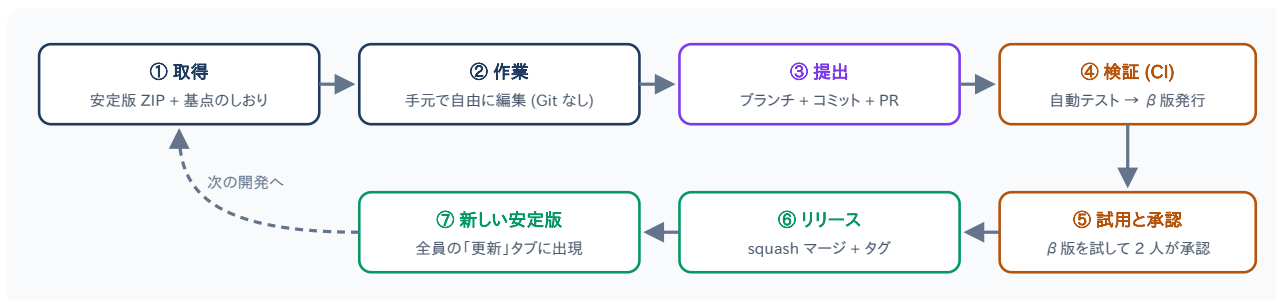


2.2 マネージャーのボタン ⇔ Git 操作 対応表

メンバーが Git を直接触ることはありません。ボタンの裏で走っているのが下の操作です。

マネージャーの操作	裏で走る Git / GitHub 操作
setup.bat (初回)	<code>git clone</code> — main を作業フォルダへ複製
マネージャー起動.bat	<code>git pull</code> — 起動のたびに最新化
起動 / 更新	Releases から ZIP 取得 + <code>version.json</code> に基点 コミット SHA を記録 (スナップショット。クローンではない)
更新版を提出	基点 SHA から ブランチ feature/名前-日付-連番 作成 → 差分を コミット → push → プルリクエスト 作成
承認 / 却下	PR レビュー (Approve / Request changes)。2 回目のクリック = 自分のレビューを dismiss
リリース	squash マージ → バージョンタグ → Release 発行 → β 版 (prerelease) 削除
最新版と統合	<code>git merge main</code> → 衝突解消 (--ours / --theirs / AI 両立マージ) → push

2.3 版のライフサイクル — 取得からリリースまで



正式版は必ず ①→⑦ の一方通行を通る。検証と承認を飛ばして main に入る道はない (ブランチ保護で強制)

3 ケーススタディ: 2 人が同時に開発したら

3.1 前提条件

メンバー X が機能 (A)、メンバー Y が機能 (B) を、どちらも安定版 v1.2 (コミット S) を基点に同時に開発しているとします。お互いの作業は知らないまま進みます。

3.2 提出時 — 衝突は原理的に起きない



- 提出 = 「基点 S と ZIP の差分を新しい **ブランチ** に **コミット** して **プルリクエスト** を作る」操作。比較相手は常に自分の基点だけ。他人と同じ行を直していても提出は成功する
- 提出時に落ちるのは **CI (pytest) の不合格・秘密情報の検出・サイズ超過** など。これは **プログラム/内容の問題** で、マージ衝突とはレイヤーが違う
- 衝突は「どちらかの枝が main に入って main が進んだ後」に、残った PR の合流可否判定 (mergeable) が再計算されて初めて出現する

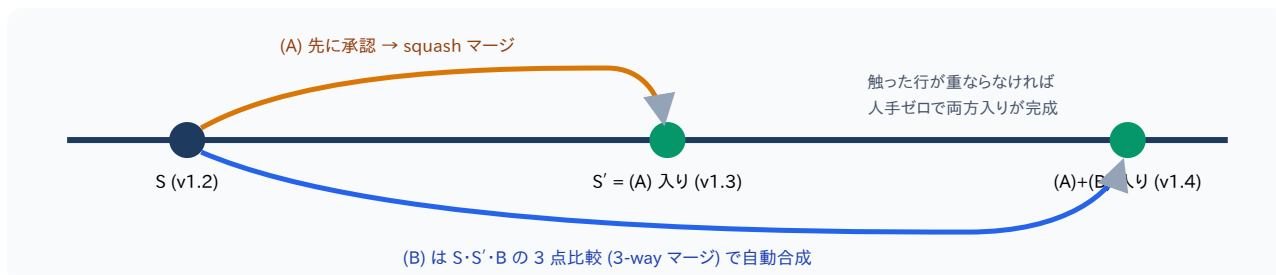
よくある誤解の整理

- 「提出でエラーが出た = 誰かとぶつかった」→ **違う**。提出時のエラーは検証 (テスト) の不合格。ぶつかり (衝突) はリリース段階の概念 (→ 3.3.2)
- 「衝突 = 誰かの作業が消える」→ **消えない**。両方の変更が保全された上で「どう混ぜるか」を人間 (提出者) が選ぶ
- 「衝突は怖い」→ 解消は 3 択 1 クリック。本当に怖いのは衝突ではなく、衝突を検出できない **フォルダ運用**

3.3 リリース時 — 枝の合流と衝突

提出が承認されてマージされる段階で、初めて 2 本の枝が出会います。結果は 2 通りだけです。

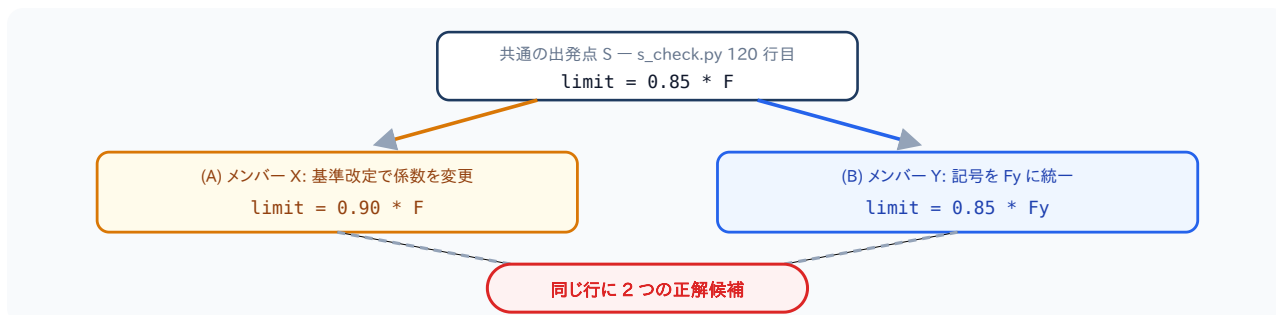
3.3.1 衝突しない場合 (別のファイル・別の行) — 3-way マージで自動合流



- ブランチ保護の `strict status checks` により、合流後の状態で CI が再実行されてからでないとマージできない
- 「文字は衝突しないが意味的に非互換」(片方が関数の引数を変え、片方が旧仕様で呼ぶ等) は Git では検出できないが、この合流後 CI が最後の網になる

3.3.2 衝突する場合 (同じ行) — 統合待ちロック → 3 択で解消

まず「衝突とは何か」を 1 枚で。



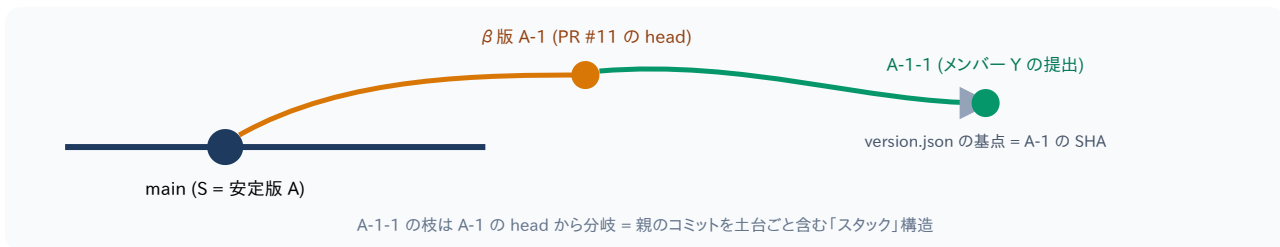
衝突の正体 = 「同じ行に対する 2 通りの変更」。どちらも業務的には正しいから、機械には選べない — 選ぶのは人間の仕事として残る
※ コード・数値・人物の行動はすべて説明用の架空の例です (実際の提出・実コードとは無関係)



- 解消は提出者の見えない作業用クローン (workrepo) 内で `git merge origin/main` を実行して行う。<<<<<<< マーカーはメンバーの目に触れない
- push で PR の中身が変わるため、全員の承認・却下を自動 dismiss。「レビューした時と違うコードがマージされる」事故を仕組みで防ぐ
- フィードバックは削除せず「以前の版へのフィードバック」として記録保持。件数カウントは統合後の新 β 版宛てで仕切り直し
- 衝突したままマージする道はない — マネージャーのリリースボタンは消え、GitHub 側もマージを拒否する (二重ガード)

4 応用: β 版を基点にした開発 (スタックド PR)

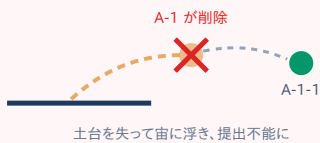
4.1 系譜は version.json の基点 SHA で自動記録される



- β 版フォルダの version.json には β 版 **ブランチ** の head SHA が入っている。これを基点に提出すると新しい枝はその SHA から分岐し、親子関係 (A $\rightarrow$ A-1 $\rightarrow$ A-1-1) が履歴に自動で残る
- マネージャーの「差分」は **基点 (A-1)** との差分だけを表示 $\rightarrow$ レビュー負担は増えない。GitHub 上の PR diff (対 main) には A-1 + A-1-1 の両方が写る

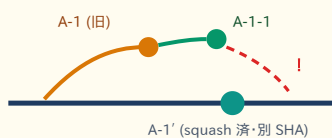
4.2 リスクは 3 つ

① 基点の消滅



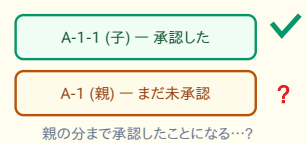
親 PR が却下確定・取り下げで消えると枝ごと削除され、基点 SHA が辿れなくなる。A-1-1 は「基点となる版が履歴に見つかりません」で**提出不能**。安定版から移植し直しになる

② 親リリース後の縮退



親が squash **マージ** されると main には「同内容だが別 SHA」が入る。子の合流はたいてい自動解決するが、autofix 等で親の最終形が違くと**衝突化しやすい** ($\rightarrow$ 統合待ちフローで解消)

③ レビュー責任の曖昧化



親が未承認のまま子を承認すると「親の内容込みの承認か」が不明瞭に。**親の決着 (リリース or 却下) を待ってから子を審査するのが原則**

5 推奨プラクティス — 結局どうすべきか

場面	推奨
基点の選び方	原則 最新の安定版 。 β 版基点 (スタック) は「親が確実にリリースされる連作」のみ
着手前	同じファイルを触りそうな作業は 一声かける 。衝突は解消できるが、しないのが一番安い
提出の粒度	小さく・こまめに 。差分が小さいほどレビューも合流も速く、衝突面積も小さい
衝突したら	怖がらず「最新版と統合」 $\rightarrow$ 迷ったら「 両方の機能を残す 」。解消後は承認が仕切り直しになるので再確認を依頼
β 版基点で作る前に	親の承認見込みを確認。 親が消えると積んだ作業ごと戻れなくなる ことを理解して積む

本資料の仕組みはすべてアプリマネージャーが自動で代行します。メンバーが覚える操作は「ZIP を選ぶ・ボタンを押す」だけです。